Aufgabe 4.5

Dokumentation - Modularität und Testbarkeit

Bei der Entwicklung von MediTrack wurde die Anwendung nach den Prinzipien des Domain-Driven Design (DDD) in mehrere fachliche Bounded Contexts aufgeteilt: Patientenverwaltung, Terminmanagement, Dokumentenverwaltung und Behandlungsmanagement. Jeder Bereich ist in einem eigenen Java-Package organisiert und besitzt eigene Domänenklassen sowie, je nach Aufgabenbereich, Services und Repositories. Dadurch sind die fachlichen Verantwortlichkeiten klar voneinander getrennt und die einzelnen Module bleiben übersichtlich und erweiterbar. Die Geschäftslogik befindet sich in den Service-Klassen, während die Repositories für die Speicherung und Bereitstellung der Daten zuständig sind. Durch Konstruktor-Injektion werden Abhängigkeiten explizit übergeben, wodurch die Komponenten unabhängig und besser testbar bleiben. Zusätzlich werden fachliche Regeln durch Validierungen abgesichert und direkte Abhängigkeiten zwischen den Bounded Contexts durch die Verwendung von IDs reduziert. Für die einzelnen Bereiche wurden automatisierte Unit-Tests mit JUnit 5 erstellt, die reguläre Abläufe, ungültige Eingaben und Fehlerfälle überprüfen; die elf Tests der Patientenverwaltung wurden nach der Einrichtung von Java 21 erfolgreich ausgeführt. Zur kontinuierlichen Integration wurde ein GitHub-Actions-Workflow eingerichtet, der Java 21 und Maven verwendet. Bei Pushes auf den Branch *main* sowie bei Pull Requests in Richtung *main* wird der Workflow automatisch gestartet und führt mit *mvn clean verify* den Build und die Tests aus. Schlägt ein Build oder ein Test fehl, wird dies im GitHub-Actions-Ergebnis angezeigt; bei erfolgreicher Ausführung wird der Build als erfolgreich markiert. Die eigenen Änderungen konnten bisher noch nicht in das gemeinsame GitHub-Repository hochgeladen werden, da dafür die erforderlichen Schreibrechte fehlen.